THE UNIVERSITY OF SYDNEY

# MTRX3760 - Lab 1
# Introduction to OOD in C++

This assignment contributes 10% towards your final mark. It must be completed in groups of two. Use the Canvas groups functionality to build your team and complete a single submission. Groups of one or three are not permitted except with prior approval from the unit coordinator. All group members must be enrolled in the same practical section.

This assignment is due before the start of your allocated lab session in two weeks, i.e. before the start of the Week 4 lab session for Friday labs, and before the start of the Week 5 lab session for Monday labs. Late assignments will be subject to the University's late submission policy. Plagiarism will be dealt with in accordance with the University of Sydney plagiarism policy. Submissions will not be accepted more than a week after the due date.

Assignments will be assessed based on the following components. Incomplete submissions will not be graded:

- **Report**: Submit a .pdf report using the class Canvas site. The report can be very simple, and needs only directly address the questions laid out in the assignment. The cover page for your report should include your group members' SIDs and practical section, do not include names.
- **Code appendix**: The appendix of your report must contain a printout in text format of all source code written for the assignment. File header comments and proper formatting will be critical to making this section readable. Inclusion as images is unacceptable.
- **Code**: Submit your code (code only, no binaries) in a .zip file via the canvas site.

You may find libreoffice writer or google or microsoft's online tools useful for preparing your report. For printing out and appending code to your report, you can use:

```
find . -name "*.cpp" -o -name "*.h" | sort -V | xargs enscript -r --color=1 -C
-Ecpp -fCourier8 -o - | ps2pdf - code.pdf
```

```
pdfunite <report name>.pdf code.pdf <concatenated report name>.pdf
```

The first command recursively finds `.cpp` and `.h` files and formats them as a pdf called `code.pdf`. The second command concatenates the code to an existing pdf file.

THE UNIVERSITY OF SYDNEY

This lab should be completed with concepts and language features taught in **Weeks 1 and 2 only**.

## A1. Robot [5 marks]

You are given two small programs that do the same thing: each models a robot that follows a line. Every cycle the robot reads its line sensor, works out how hard to steer, and sets its two drive motors.

Extend both programs to give the robot a battery. The battery starts fully charged, loses charge every cycle, and once it drops below 80 the robot's forward speed is reduced from 0.5 to 0.25. Steering is unaffected. Edit the two source files directly; keep the behaviour of the two versions equivalent so the comparison is fair.

In your report show the output of both versions of your code. The program output should be copy/pasted into your report as text, not displayed as a screenshot.

In your report, answer the following. These should be brief responses, one or two sentences each.

1. When you added the battery, what changed in main() in each version?
2. In the function-based version, what does main() have to know about the robot's parts? What does it know in the object-based version?
3. What is the name of the object-oriented principle that produced this saving?

THE UNIVERSITY OF SYDNEY

## A2. Oven [5 marks]

You are given two small programs that do the same thing: each models two ovens that warm up in steps and warn if overheated. One stores their temperatures as public data and accesses them directly; the other keeps them private and works through meaningful functions.

Both currently store temperature as a whole number of degrees Celsius. Change both so that the temperature is stored internally as tenths of a degree, for finer resolution. The behaviour and printed output should stay the same. Edit the two source files directly.

In your report show the output of both versions demonstrating correct operation. The output should be copy/pasted into your report as text, not shown as a screenshot.

In your report, answer the following. One or two sentences each.

1. Which version required more changes? Name the functions or lines you had to edit in each.
2. What is the name of the object-oriented principle that kept the change contained in one version?
3. Did main() need to change in the public version? In the private version? Explain the difference.

THE UNIVERSITY OF SYDNEY

## A3. Clock [5 marks]

You are given two identical copies of a program modelling a simple clock that keeps a time in minutes and can advance, report, and reset. Your job is to make two extended versions of this program, both of which add an alarm clock. An alarm clock lets you set an alarm time and check whether it is ringing.

Build two versions:

- In clock_standalone.cpp, build the alarm clock as a brand-new, separate class. Do not use inheritance.
- In clock_inherit.cpp, build the alarm clock by inheriting from CClock. In this second version do not modify CClock, and keep your changes to main to a minimum.

Edit the two source files directly. In both, create and use an alarm clock alongside the plain clock already in main: set its alarm, advance it until it rings, and report it.

In your report show the output of both versions demonstrating correct operation. The output should be copy/pasted into your report as text, not shown as a screenshot.

In your report, answer the following. One or two sentences each.

1. In the standalone version, which parts of the clock's behaviour did you have to repeat?
2. In the inheritance version, what did you add, and what did you get without writing it?
3. What is the name of the object-oriented principle that let the second version reuse the clock's behaviour?

THE UNIVERSITY OF SYDNEY

## A4. Power Budget [5 marks]

You are given two versions of a program. Each models a set of devices on a robot: a motor and an LED. Each reports how much power each device is drawing, along with the total. The two versions are written differently but produce the same output.

Your job is to add a new kind of device to both versions: a heater, which draws power according to a heat setting you give it. In each version, add your heater as a new class without modifying CDevice. Create and use a heater alongside the devices already in main.

In your report show the output of both versions demonstrating correct operation. The output should be copy/pasted into your report as text, not shown as a screenshot.

In your report, answer the following. One or two sentences each.

1. What did you have to change in each version to add the heater? Answer for both versions separately.
2. Which version was easier to extend, and what specifically made it easier?
3. What is the name of the object-oriented principle behind that difference?

THE UNIVERSITY OF SYDNEY

## A5. Robot Controller [30 marks]

Using the lessons learned above, design a robot controller that runs a set of subsystems. Partner A builds the subsystems, Partner B builds the controller, and both partners work on integration and testing.

Your controller must have at least two different kinds of subsystem, for example a drive motor and a line detector, each with its own internal state. Design with the idea that many more subsystems might get added later.

The controller must, on each cycle, run every subsystem and report its state. Run the program for at least three cycles so the subsystems' changing state is visible in the output.

Your code should be split across multiple files, following the principle of one class per file.

In your report show the output of your program. The output should be copy/pasted into your report as text, not shown as a screenshot.

In your report, answer the following. One or two sentences each.

1. What did Partner B's controller need to know about the subsystems? What did Partner A's subsystems need to know about the controller?
2. What did you have to change, adapt, or correct to make the two halves work together?
3. Did you design the testing before or after the subsystems and integrated system? In future, which option do you think would be the most time-efficient?
4. In which places does your design, including test code, use the following concepts? Be specific, point to specific parts of the program:
    a. Modularity
    b. Encapsulation
    c. Inheritance
    d. Polymorphism

## Design and Code Quality

**Design Quality [25 marks]**
A5 will be assessed on design principles and best practices.

**Code Quality [25 marks]**
Your code will be assessed on style and coding best practices. Your tutor will annotate the code in the appendix, be sure it’s included and legible.

## Self-Assessment Bonus

Self-assessment: in your report, estimate your mark for each component of the lab, and your total out of 100. Ignore late penalties, and do not include the self-assessment bonus in your estimate. If your estimated total is within 5 marks of your actual total, you earn the 5 self-assessment marks. The bonus can raise your total above 100, to a maximum of 105.

For example:

```
A1          4 / 5
A2          3 / 5
A3          4 / 5
A4          3 / 5
A5          20 / 30
Design      18 / 25
Code        18 / 25
Total:      70 / 100
```

## Design Checklist

When designing your own program for A5, here is a set of checks to consider:

- Is the program broken into classes that make sense and closely reflect the structure of the problem being solved?
- Are there the right number of instances of each class?
- Are the relationships between classes meaningful? Does your design use encapsulation / nesting appropriately?
- Are there any global functions or variables* or code in main that could be encapsulated inside a class instead?
- Is member data closely related to the class that contains it?
- Are member functions inside classes that they most logically belong to?
- Are classes responsible for operating on their own data, and not the data of other classes?
- Do classes ask other classes to do things by calling their member functions?
- Do member functions correspond to logical, atomic, easy to understand operations?
- Are all member variables *private*?
- Do classes own and operate on all important data, or is there important data being passed around as function arguments as we do in functional programming?
- Is your submission complete? Review the first page of this handout.
- Are declarations separate from implementations?
- Is there one class per header file?
- Could a new subsystem be added without changing existing code?
- Where you have used inheritance, is it a genuine "is a" relationship?

*For this assignment you may have global consts, but there should be no global functions or variables, and no substantial code in main.

THE UNIVERSITY OF SYDNEY

# Coding Style Guideline

This is a summary of specific advice given in lectures, but be mindful that you are making many more decisions than these about how to write your code. Throughout the lectures we have emphasised consistency: the code you submit should use the same style throughout.

Lec 1A

- Rather than “using namespace” use the “std::” syntax

Lec 1B

- Informative naming
- Consistent, defensive bracing; one statement per line
- Consistent indentation, spacing, bracing
- Readable variable declarations, one variable per line
- Avoid #defines, they hide bugs well; use consts instead
- Comments are mandatory for good code, see the lecture notes for 6 important spots to check
- Avoid comments that are obvious
- Shape comments & whitespace to highlight structure
- Only use the “this” pointer when necessary

Lec 2A, 2B

- Emphasise parallel structures to help spot bugs
- Avoid repeating code: fewer bugs, less code to maintain
- Avoid multiple exits, these make it harder to debug with breakpoints, couts